Parameter-Efficient Continual Learning for Text Classification

A class-incremental study of LoRA, bottleneck adapters and experience replay on a 4 GB GPU

Marcin Czarnecki

M.Sc. Computer Science — Artificial Intelligence
Continual Learning course project

University of Pisa, A.Y. 2025/2026

Abstract

This report studies parameter-efficient continual learning (CL) for intent classification under the class-incremental learning (CIL) scenario, in which a single model must learn a stream of disjoint groups of classes and, crucially, classify test inputs without being told which task they belong to. I freeze a pre-trained RoBERTa-base encoder and compare seven training regimes built from one configurable pipeline: a naive full fine-tuning lower bound, a frozen linear probe, sequential LoRA, sequential bottleneck adapters, LoRA combined with experience replay (the proposed method, lora_er), a joint-training upper bound, and a per-task Task-IL oracle. The benchmark is Banking77 partitioned into seven tasks of eleven classes each.

On the full benchmark, lora_er reaches 0.820 average accuracy, recovering roughly 89% of the joint upper bound (0.917) and dramatically outperforming every method without replay, which collapse to 0.12–0.17 accuracy through catastrophic forgetting. Three ablations on the proposed method show that (i) a small LoRA rank is sufficient — accuracy plateaus from rank 4 to 16 and degrades at rank 32; (ii) replay buffer capacity is the dominant lever, with accuracy rising monotonically from 0.583 at capacity 100 to 0.857 at capacity 1000; and (iii) the method is robust to task ordering (0.835 ± 0.008 over three class orderings). The entire study runs in about 3.3 hours on a single 4 GB laptop GPU with peak memory below 2.7 GB, demonstrating that competitive continual learning for NLP is accessible on modest hardware. A dedicated section documents the engineering and numerical problems encountered and how each was resolved.

1. Introduction

Neural networks trained on a sequence of tasks tend to overwrite previously acquired knowledge when they learn new data, a failure mode known as catastrophic forgetting. Continual learning aims to train a single model on a non-stationary stream while retaining performance on all tasks seen so far. This project investigates the problem in the context of text classification, where large pre-trained transformer encoders are the de-facto backbone but full fine-tuning per task is expensive in both computation and storage.

Two ideas are combined. First, parameter-efficient fine-tuning (PEFT) — LoRA and bottleneck adapters — keeps the backbone frozen and trains only a tiny number of additional parameters, which limits the capacity available to overwrite old knowledge. Second, experience replay stores a small sample of past examples and interleaves them with new data, directly counteracting forgetting. The central question is whether PEFT alone mitigates forgetting, or whether replay is the decisive ingredient.

The study is deliberately constrained to a single consumer GPU with 4 GB of memory. This constraint is not incidental: it reflects the realistic setting of a practitioner without datacenter resources and forces every design choice to respect a tight memory budget. The main contributions are:

- A single, configuration-driven pipeline that realises seven CL regimes — from lower bounds to upper bounds — so that all methods are compared under identical data, splits and evaluation.

- A clean empirical result on Banking77 under the demanding CIL scenario, isolating experience replay as the decisive factor and quantifying how close the proposed method comes to the joint upper bound.
- Three controlled ablations (LoRA rank, replay buffer capacity, task ordering) that characterise the proposed method and justify its hyper-parameters.
- A reproducible engineering account, including a candid record of the numerical and dependency problems encountered on the target hardware and their resolutions.

2. Background and Related Work

2.1 Continual learning scenarios

Continual learning is commonly divided into three scenarios. In task-incremental learning (Task-IL) the task identity is provided at test time, so the model only chooses among the classes of the known task. In domain-incremental learning the label space is fixed but the input distribution shifts. In class-incremental learning (CIL) new classes arrive over time and, at test time, the model must discriminate among all classes seen so far without any task identifier. CIL is the hardest and most realistic of the three, because the model must resolve interference between classes that were never seen together during a single training phase. This project targets CIL; a Task-IL oracle is included only as an upper reference.

2.2 Parameter-efficient fine-tuning

LoRA (Low-Rank Adaptation) freezes the pre-trained weights and injects trainable low-rank update matrices into the attention projections, so that only a small fraction of parameters is learned per task. Bottleneck adapters insert small down-projection / non-linearity / up-projection modules between transformer sub-layers and train only those. Both keep the backbone fixed, which is attractive for continual learning because the shared representation cannot drift arbitrarily; however, as the experiments show, limiting plasticity does not by itself prevent forgetting at the classifier level.

2.3 Rehearsal and experience replay

Rehearsal methods retain a bounded memory of past examples and replay them while learning new tasks. Reservoir sampling maintains a uniform random sample of an unbounded stream in fixed memory, which makes it a natural fit for an online task sequence. Experience replay is simple, architecture-agnostic, and — as confirmed here — highly effective even with a modest buffer. The proposed method combines LoRA adaptation with a reservoir replay buffer.

2.4 Evaluation of continual learners

Following the standard protocol, results are summarised by the accuracy matrix R, where R[i][j] is the test accuracy on task j after training through task i. Average accuracy (ACC) is the mean of the final row over all tasks; Backward Transfer (BWT) measures how learning later tasks affects earlier ones; and Average Forgetting measures the drop from the best historical accuracy of each task to its final accuracy. Forward Transfer is omitted because, under a single growing classification head in CIL, the accuracy of a task before it has been learned is not well defined.

3. Methodology

3.1 Problem setting

The model observes a stream of T tasks. Each task introduces a disjoint set of new classes and is trained on once, in order, with no revisiting of a task's full training set (a small replay memory aside). Evaluation is class-incremental: at test time no task identifier is given and the prediction is the arg-max over every class seen so far. This single-head, no-task-id protocol is applied identically to all methods so that differences in the results reflect the learning strategy rather than the evaluation.

3.2 Frozen backbone and cosine classification head

The encoder is RoBERTa-base (about 124.6 million parameters), kept entirely frozen in every method except the naive full fine-tuning lower bound. Sequences are tokenised to a maximum length of 64. Classification uses a cosine-similarity head: input features and per-class weight vectors are L2-normalised and compared by cosine similarity scaled by a fixed temperature. The head grows as new classes arrive, and the expansion preserves the weight rows of previously seen classes. Compared with a nearest-class-mean classifier the cosine head avoids centroid drift as the representation changes, and its normalisation reduces the recency bias that otherwise inflates scores for the most recently trained classes.

3.3 PEFT modules

LoRA is applied to the query and value projections of every attention block, with rank r = 8 and scaling alpha = 16 in the main experiments. Because the model uses its own external classification head rather than the transformer's sequence-classification head, LoRA is configured in feature-extraction mode. The bottleneck adapter is a custom Houlsby-style module attached via forward hooks, with the up-projection zero-initialised so that an untrained adapter is an identity map at the start of each task. Using forward hooks rather than a third-party adapter library was a deliberate choice forced by a dependency conflict, discussed in Section 7.

3.4 Experience replay

The replay buffer is a reservoir that stores tokenised input ids (not cached embeddings) together with their global labels. Storing token ids rather than embeddings is essential: under LoRA the encoder's output for a stored example changes as adaptation proceeds, so cached embeddings would become stale and misrepresent the past distribution, whereas re-encoding stored token ids always reflects the current model. During training on a new task, mini-batches are augmented with samples drawn from the buffer. The default buffer capacity in the main experiments is 500.

3.5 Methods compared

All seven regimes are produced by one pipeline through a single configuration switch. They span the full range from lower to upper bounds:

Method	Strategy	Backbone	Adaptation	Replay
naive_ft	sequential	trainable	full fine-tune	no
linear_probe	sequential	frozen	head only	no
sequential_lora	sequential	frozen	LoRA	no
sequential_adapter	sequential	frozen	bottleneck adapter	no
lora_er (proposed)	sequential	frozen	LoRA	yes (cap 500)
joint	joint (upper bound)	frozen	LoRA, all data at once	n/a
per_task_oracle	Task-IL reference	frozen	separate adapter per task	n/a

PEFT methods are trained with learning rate 1e-3 for 3 epochs per task; the naive full fine-tuning baseline uses a lower learning rate of 2e-5 for 5 epochs so that it genuinely learns each task before forgetting it, making it an honest lower bound rather than a model that simply fails to train.

3.6 Evaluation metrics

After each task the full accuracy matrix R is recorded by evaluating on the held-out test set of every task seen so far. From R we report ACC (mean of the final row), Average Forgetting and BWT. Average accuracy over the stream — the mean of the filled part of each row of R — is also plotted to visualise the trajectory of knowledge retention as tasks accumulate.

4. Experimental Setup

4.1 Dataset

The primary benchmark is Banking77, a fine-grained banking intent-classification dataset of 77 classes. It is partitioned into seven tasks of eleven classes each (a 7×11 CIL split), giving roughly 117 training examples per class after a validation split is carved out. A small AG News split (2 tasks of 2 classes) is used only as a fast sanity benchmark during development, with per-class caps to keep it quick.

4.2 Hardware and software

All experiments run on a single NVIDIA GeForce GTX 1650 Ti laptop GPU with 4 GB of VRAM (Turing architecture, compute capability 7.5) under Windows 11. Mixed-precision training uses fp16; bf16 is unsupported on this architecture (see Section 7). The software stack is pinned for reproducibility: Python 3.11.9, PyTorch 2.5.1 (CUDA 12.4 build), transformers 4.46.3, peft 0.13.2, datasets 3.1.0, numpy 1.26.4 and pyarrow 16.1.0. The specific reasons for several of these pins are part of the engineering story in Section 7.

4.3 Training details

Unless stated otherwise, results use a fixed random seed (42), LoRA rank 8 with alpha 16, replay capacity 500, maximum sequence length 64, and the per-method epoch and learning-rate settings of Section 3.5. Peak GPU memory is recorded per task to confirm that every configuration stays within the 4 GB budget.

4.4 Phased development methodology

The system was built in seven incremental phases, each with an explicit definition of done and an automated test suite (28 unit tests in total) that had to pass before proceeding. This discipline kept regressions out of a code base that grew steadily in complexity and made the many environment problems tractable, because each problem could be isolated to the phase that surfaced it:

1. **Phase 0 —** project scaffolding, environment setup and a smoke test (peak VRAM 502 MB).
2. **Phase 1 —** the data pipeline: loading, the CIL task-stream construction for Banking77 and AG News, tokenisation and split validation.
3. **Phase 2 —** models: the frozen backbone, the growable cosine head, LoRA and the custom bottleneck adapter, behind one build_model factory. A single-task LoRA check reached 0.97 accuracy training only 0.24% of parameters.
4. **Phase 3 —** the reservoir replay buffer and the continual trainer that fills the accuracy matrix R, plus the method registry. A sanity run showed sequential LoRA forgetting task 0 to 0.000 while replay recovered it.
5. **Phase 4 —** the unified runner exposing all seven methods from a single switch, including the joint upper bound and the Task-IL oracle implemented honestly.
6. **Phase 5 —** the metrics (ACC, Forgetting, BWT), the experiment runner and the full Banking77 main table.
7. **Phase 6 —** figure generation and the three ablation studies, together with a fix for a VRAM-accumulation bug that only manifested when many runs executed back to back.

5. Results

5.1 Main comparison

Table 2 reports the final metrics for all seven methods on the full Banking77 CIL stream. The single proposed method that uses replay, lora_er, is the only realistic approach that retains its knowledge; every method without replay collapses.

Method	ACC	Forgetting	BWT	Peak VRAM (MB)
per_task_oracle	0.975	—	—	1806
joint (upper bound)	0.917	—	—	1807
lora_er (proposed)	0.820	0.168	-0.168	1938
naive_ft	0.171	0.946	-0.946	2465
sequential_lora	0.151	0.966	-0.966	2285
sequential_adapter	0.147	0.969	-0.969	2650
linear_probe	0.119	0.825	-0.825	1499

Table 2. Final results on Banking77 (7×11 CIL, seed 42), sorted by accuracy.

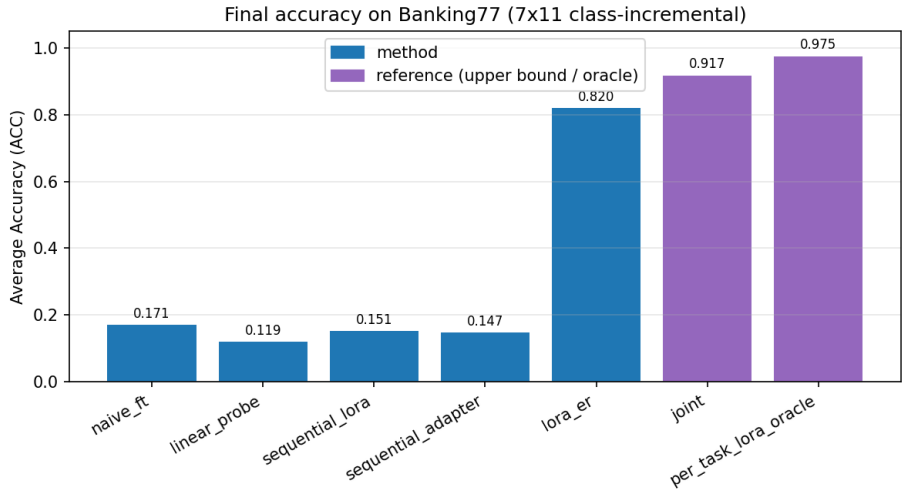


Figure 1. Final accuracy per method. References (joint, oracle) are shown in a distinct colour.

The result is unambiguous. The proposed lora_er recovers about 89% of the joint upper bound while training strictly sequentially and seeing each task only once plus a small replay memory. The four methods without replay land between 0.119 and 0.171, an order of magnitude worse, despite learning each individual task almost perfectly at the time (their R-matrix diagonals are 0.97–0.99). Notably, the choice of PEFT module barely matters without replay: sequential LoRA (0.151) and sequential adapters (0.147) collapse almost identically, confirming that the decisive ingredient is replay, not the adaptation mechanism. The per-task oracle (0.975), which is granted the task identity at test time, marks the easy Task-IL ceiling; the gap between it and lora_er is the price of the realistic no-task-id CIL setting, most of which the proposed method closes.

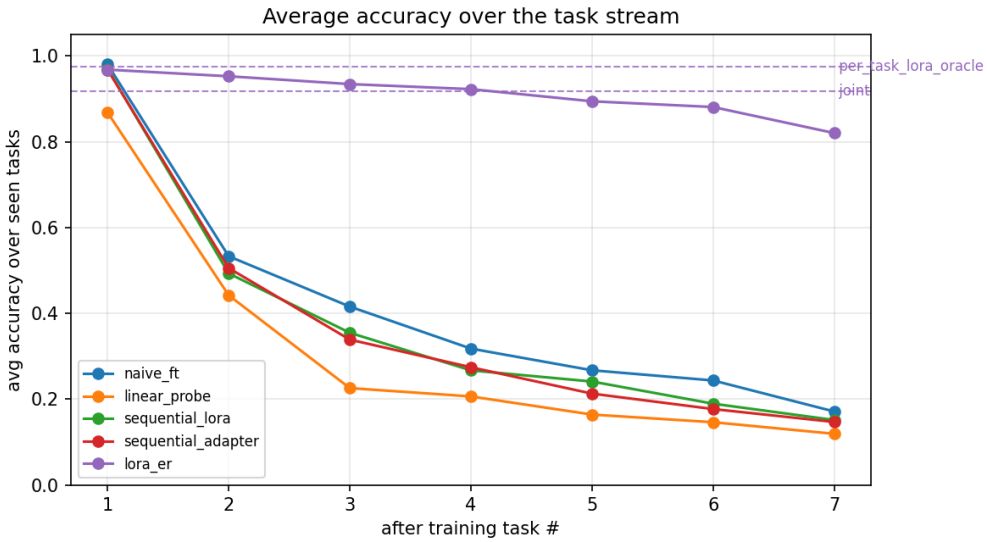


Figure 2. Average accuracy over seen tasks as the stream progresses. Replay (purple) stays close to the upper bounds; every no-replay method decays steeply.

Figure 2 shows the mechanism over time. Methods without replay lose accuracy steeply with each new task as old classes are overwritten, while lora_er tracks just under the joint and oracle reference lines across the whole stream. Figure 3 quantifies the endpoint of this process: average forgetting is 0.83–0.97 for the no-replay methods versus 0.168 for lora_er.

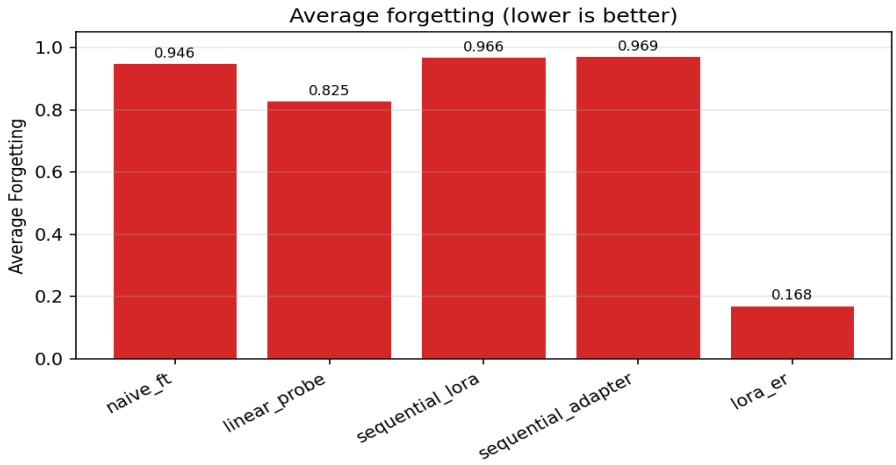


Figure 3. Average forgetting (lower is better). Note that linear_probe's apparently lower forgetting is an artefact of its lower per-task accuracy, not better stability.

A subtle point deserves emphasis: linear_probe shows a lower forgetting value (0.825) than the LoRA and adapter baselines (about 0.97), but this must not be read as greater stability. Its per-task accuracy is itself low (frozen features classify each task at only 0.68–0.88 on the diagonal), so it simply has less accuracy to lose. Forgetting is only meaningful alongside ACC, and linear_probe has the lowest final accuracy of all (0.119).

5.2 Forgetting dynamics

The accuracy matrices make the contrast concrete. Figure 4 (sequential LoRA, no replay) shows a bright diagonal — each task is learned well — but the off-diagonal entries collapse to near zero as soon as later tasks arrive, with only a faint one-step recency memory. Figure 5 (the proposed lora_er) keeps the whole lower triangle bright: old tasks remain at 0.67–0.97 accuracy even after six further tasks. The two heatmaps share a colour scale, so the difference is the visual signature of forgetting versus retention.

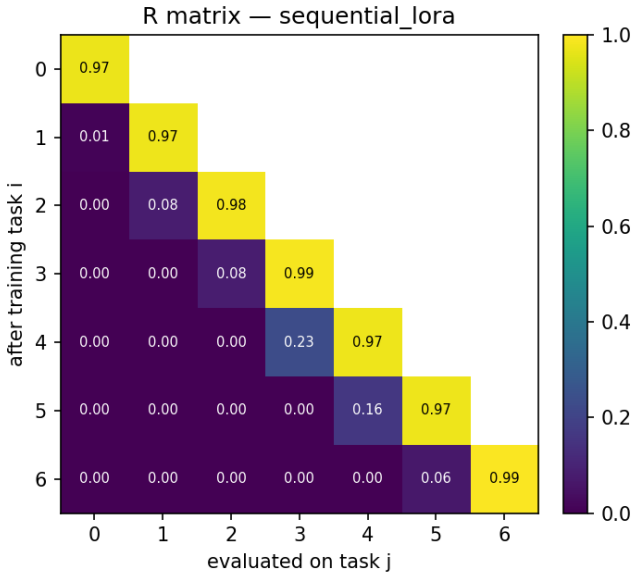


Figure 4. Accuracy matrix R for sequential LoRA (no replay): catastrophic forgetting — the off-diagonal vanishes.

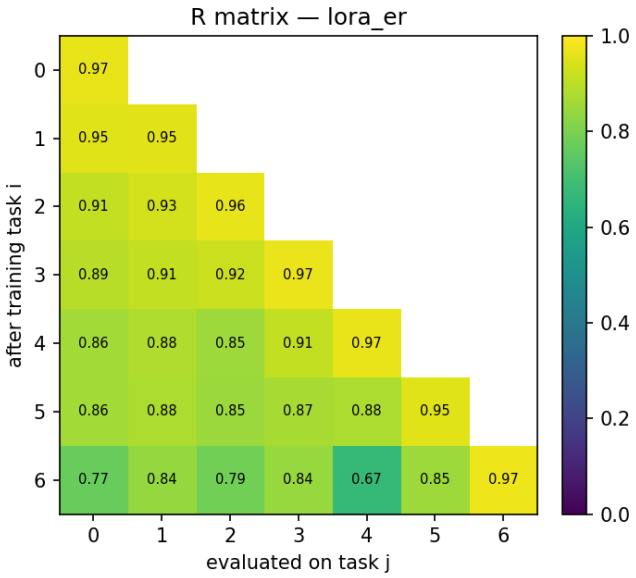


Figure 5. Accuracy matrix R for lora_er (LoRA + replay): old tasks are retained throughout the stream.

6. Ablation Studies

Three ablations were run on the proposed lora_er, each varying one factor on the full benchmark while holding the rest at the main-experiment settings.

6.1 LoRA rank

The LoRA rank r was varied across {2, 4, 8, 16, 32} with the scaling fixed at alpha = 2r so that the effective LoRA scale is constant and the rank-8 point reproduces the main configuration.

LoRA rank r	2	4	8	16	32
ACC	0.809	0.827	0.827	0.833	0.819
Forgetting	0.184	0.162	0.162	0.162	0.172

Table 3. LoRA rank ablation on lora_er.

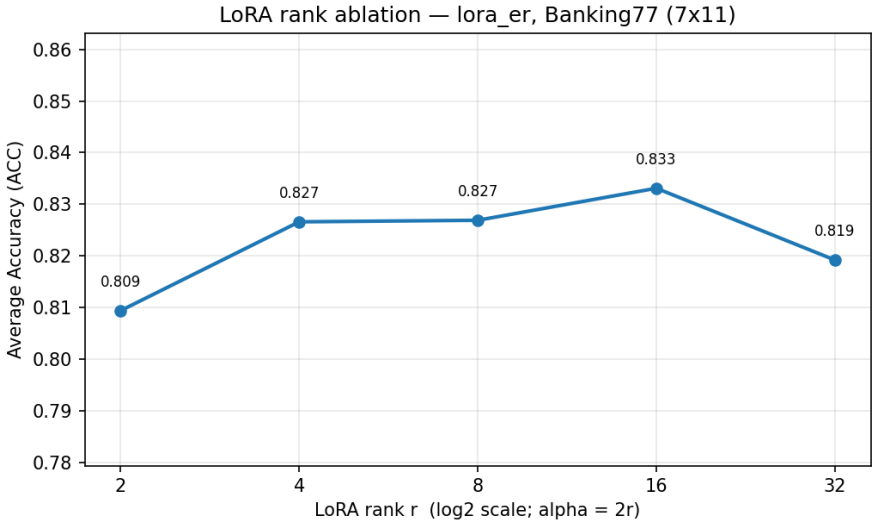


Figure 6. Accuracy versus LoRA rank. Performance plateaus between rank 4 and 16 and degrades at rank 32.

Accuracy rises sharply from rank 2 to rank 4, then plateaus through ranks 4, 8 and 16, and slightly declines at rank 32. A small rank is therefore sufficient, and the main-experiment choice of rank 8 sits safely on the plateau. The slight drop at rank 32 indicates that excess adapter capacity is not merely wasteful but mildly harmful in the sequential setting: a larger low-rank update has more freedom to drift away from earlier tasks. Peak memory is essentially flat across ranks, so rank carries a negligible memory cost.

6.2 Replay buffer capacity

The buffer capacity was varied across {100, 200, 500, 1000}.

Buffer capacity	100	200	500	1000
ACC	0.583	0.715	0.833	0.857
Forgetting	0.455	0.302	0.156	0.124

Table 4. Replay buffer-size ablation on lora_er.

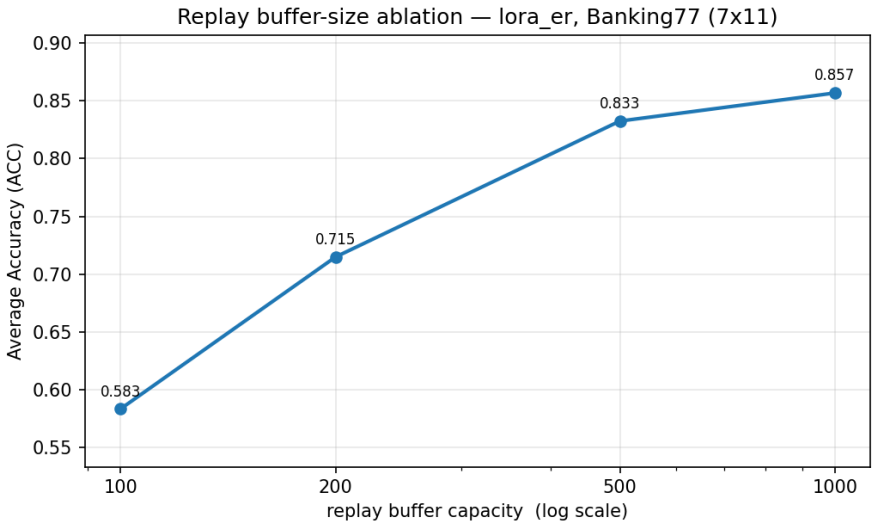


Figure 7. Accuracy versus replay buffer capacity (log scale). Capacity is the dominant lever, with diminishing returns.

Buffer capacity is the single most influential factor in the whole study. Accuracy climbs monotonically from 0.583 at capacity 100 to 0.857 at capacity 1000, while forgetting falls from 0.455 to 0.124. The gain is largest from 100 to 500 and clearly diminishing from 500 to 1000, but performance has not fully saturated at 1000. The buffer stores token ids on the host, so larger capacities do not increase GPU memory. This ablation localises the cost of continual learning precisely: it is paid in memory budget for past examples, and the proposed method converts that budget into retained accuracy very efficiently.

6.3 Task-order sensitivity

To verify that the headline result is not an artefact of one particular ordering of classes into tasks, lora_er was rerun with three different class-ordering seeds (the same training seed throughout).

Class-order seed	0	1	2	mean ± std
ACC	0.824	0.843	0.838	0.835 ± 0.008

Table 5. Task-order sensitivity of lora_er.

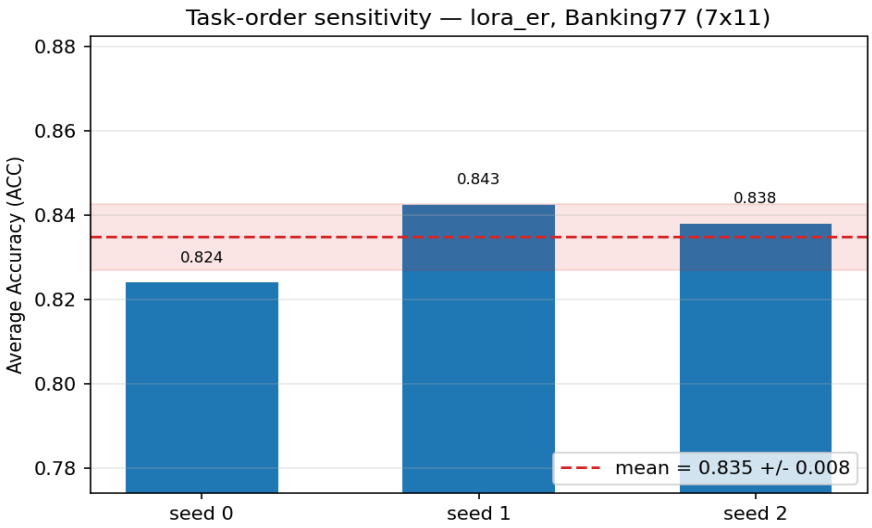


Figure 8. Accuracy across three class orderings, with mean and standard-deviation band.

The standard deviation across orderings is below one accuracy point (0.835 ± 0.008), and the main-table point (seed 0) reproduces within noise. The proposed method is therefore robust to task ordering, and the single-seed main table is a faithful summary rather than a lucky draw. These three runs also serve as the error bars for the headline method.

7. Implementation Challenges and Lessons Learned

A substantial part of the effort went into making a modern deep-learning stack run correctly on a 4 GB Turing laptop GPU under Windows. The problems below are reported in full because they shaped design decisions and because each carries a transferable lesson. They are roughly in the order they were encountered.

7.1 CUDA driver newer than any matching PyTorch build

The installed driver exposed CUDA 13.2, for which no PyTorch 2.5.1 wheel existed. Installing from the CUDA 12.4 wheel index resolved it: NVIDIA drivers are backward compatible, so a cu124 build runs against a 13.x driver. Lesson: match the PyTorch build to an available wheel, not to the exact driver version; the driver only needs to be at least as new as the build's toolkit.

7.2 System Python too new for the wheels

The system Python was 3.14, for which the required torch 2.5.1 wheels were not published. The fix was to create a dedicated virtual environment on Python 3.11.9. Lesson: on Windows especially, pin the interpreter version to one with mature wheels for the whole stack before installing anything.

7.3 Adapter library versus transformers conflict

The third-party adapters library (version 1.0.1) pinned transformers to roughly 4.45.2, which conflicted with the 4.46.3 required elsewhere. Rather than downgrade the whole stack, the bottleneck adapter was reimplemented as a small custom module attached through forward hooks, with a zero-initialised up-projection. This removed the dependency entirely and gave full control over the adapter's behaviour. Lesson: a focused 60-line reimplementation can be preferable to bending the rest of the environment around one library's version constraints.

7.4 A native crash from a pyarrow / numpy ABI mismatch

Reading the cached Arrow files produced by the datasets library caused a hard Windows access-violation crash with no Python traceback. The cause was an ABI mismatch: a pyarrow built against numpy 2 was loaded alongside numpy 1.26.4. Because the failure was in native code, it was diagnosed by enabling the Python fault handler to capture the C-level stack, then resolved by pinning pyarrow to 16.1.0 (numpy-1 compatible) and clearing the stale datasets cache. Lesson: silent native crashes usually mean a binary ABI conflict between compiled packages; the fault handler is the right tool, and numpy major-version compatibility must be checked across every compiled dependency.

7.5 bf16 falsely reported as supported on Turing

The framework's bf16-support query returned true on the Turing GPU even though Turing has no real bf16 acceleration, which led to unstable behaviour. The fix was to force fp16 explicitly by checking the GPU's compute capability and rejecting bf16 below the architecture that introduced it. Lesson: capability queries can be optimistic; gate mixed-precision on the actual compute capability rather than a convenience flag.

7.6 A sanity benchmark that ran for hours

The AG News sanity check was intended to be fast but, at full size (tens of thousands of examples per task), it ran for hours and defeated its purpose. Per-class caps were added to the data configuration so the sanity stream completes in about a minute. Lesson: a sanity test must be small by construction; make subsampling a first-class configuration option from the start.

7.7 A .gitignore pattern that silently dropped source code

A data/ entry in .gitignore matched the src/data/ package as well as the intended dataset directory, silently excluding part of the data pipeline from version control. Anchoring the pattern to /data/ (and /experiments/) fixed it. Lesson: unanchored gitignore patterns match at any depth; always anchor directory ignores with a leading slash and verify with a dry-run status.

7.8 GPU memory accumulating across successive runs

When many training runs executed back to back — as in the ablation sweeps — peak VRAM climbed steadily from run to run (observed rising by roughly 0.5 GB per run, toward the 4 GB ceiling), even though each run's configuration was comparable. The cause was reference cycles created by the PEFT and adapter forward hooks: the reference-counting collector alone did not free a finished model promptly, so models accumulated on the device. The fix was to force a garbage collection and empty the CUDA cache after every run and after every per-task model in the oracle loop. With the fix, peak memory stayed flat across an entire sweep (about 1.93 GB per run). This bug surfaced only under the quick ablation, which is precisely why a cheap end-to-end smoke test before the long unattended runs was worth the time. Lesson: in long-lived processes that build many models, free them explicitly between runs; cyclic references plus a caching allocator will otherwise leak device memory and eventually cause an out-of-memory failure.

Beyond these specific defects, two practices proved their worth repeatedly: the phased build with a passing test suite at every gate, which localised each failure to the component that introduced it; and the habit of running a fast, subsampled end-to-end pass before committing

to multi-hour experiments, which caught the memory leak before it could waste a night of compute.

8. Conclusion

This project set out to determine what makes parameter-efficient continual learning work for text classification under the demanding class-incremental scenario, and to do so within a 4 GB GPU budget. The answer is clear: experience replay is decisive. With a frozen RoBERTa-base encoder and only LoRA adaptation, the proposed lora_er reaches 0.820 average accuracy on Banking77 — about 89% of the joint upper bound — while every method without replay collapses below 0.18 through catastrophic forgetting. The choice between LoRA and bottleneck adapters is, by comparison, almost irrelevant in the absence of replay.

The ablations sharpen this picture. A small LoRA rank suffices and excess rank slightly hurts; replay buffer capacity is the dominant performance lever with smooth, diminishing returns; and the method is robust to the order in which classes arrive. Together these justify the main configuration and show that the cost of continual learning here is paid primarily in the memory budget allocated to past examples.

The study has clear limitations. It uses a single benchmark and a single backbone, the main table is reported at one training seed (with task-order error bars provided only for the proposed method), and replay is the only anti-forgetting mechanism examined. Natural extensions are to add multi-seed error bars for every method, to compare against regularisation-based continual learning such as elastic weight consolidation, to test additional benchmarks and longer task streams, and to combine replay with knowledge distillation. Nonetheless, the central, practical message stands: competitive class-incremental learning for NLP is achievable on a modest laptop GPU, and a small replay buffer is the most cost-effective ingredient for getting there.